

PRACTICAL-3

AIM: Implementation of max-heap sort algorithm.

ALGORITHM:

Max-Heap sort :-
Algorithm:-
Input = Array of element
Output = Sorted element
Step 1 $\rightarrow$ Start
Step 2 $\rightarrow$ For $i = n/2$ to 1, do
Step 3 $\rightarrow$ largest = i
Step 4 $\rightarrow$ $i = 2^x i$
Step 5 $\rightarrow$ $r = 2^x i + 1$
Step 6 $\rightarrow$ if $1 \leq n$ and $arr[1] > arr[largest]$
then do largest = 1
Step 7 $\rightarrow$ if $r \leq n$ and $arr[r] > arr[largest]$
then do largest = r
Step 8 $\rightarrow$ If largest $\neq i$ then do
Step 9 $\rightarrow$ swap $arr[largest]$ and $arr[i]$
Step 10 $\rightarrow$ For (int $i = size$; $i > 1$; $i--$)
Step 11 $\rightarrow$ swap $arr[1]$ and $arr[i]$
Step 12 $\rightarrow$ max heapify ($arr, i-1, 1$)
Step 13 $\rightarrow$ stop -

CODE:

```
#include <iostream>
using namespace std;

int heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if(left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
    return 0;
}

int swap(int arr[], int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
    return 0;
}

int heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    for (int i = n - 1; i >= 0; i--) {
        swap(arr, 0, i);
        heapify(arr, i, 0);
    }
    return 0;
}

int main() {
    int n;
    cout << "Enter the size of the array: ";
    cin >> n;

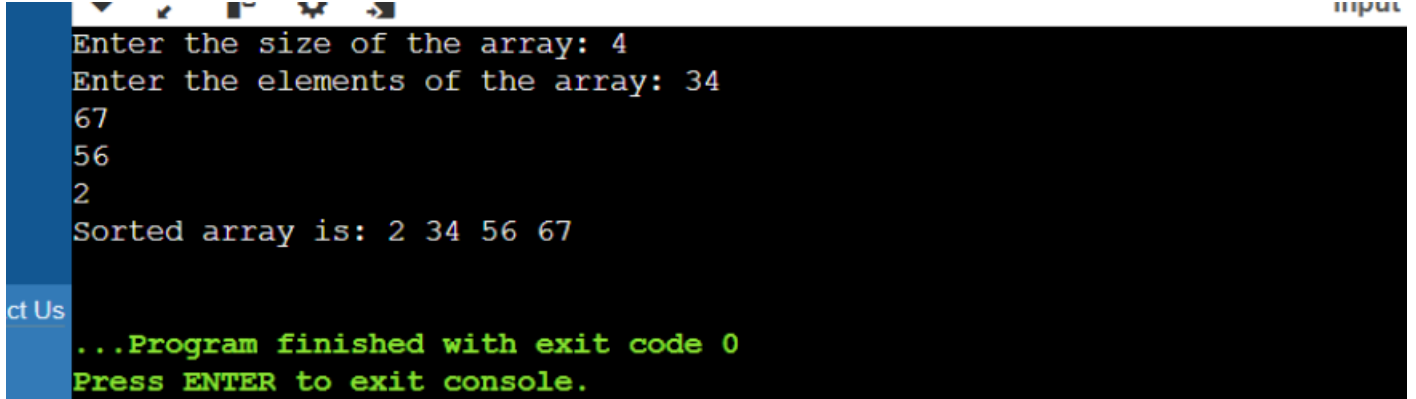
    int arr[n];
    cout << "Enter the elements of the array: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    heapSort(arr, n);
}
```

```
cout << "Sorted array is: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}
```

OUTPUT:



The screenshot shows a terminal window with the following text:

```
Enter the size of the array: 4
Enter the elements of the array: 34
67
56
2
Sorted array is: 2 34 56 67

...Program finished with exit code 0
Press ENTER to exit console.
```

TIME COMPLEXITY:

Time Complexity

For $(i = n/2; i > 1; i--) \rightarrow n$ multiplications
 $\text{maxHeapify(arr, n, i)} \rightarrow \log n$
 For $(i = n; i > 1; i--) \rightarrow n$
 $\text{swap(arr[i], arr[1])} \rightarrow 1$
 $\text{MaxHeapify(arr, i-1, 1)} \rightarrow \log n$
 $T(n) = n \times \log n + n \times 1 \times \log n$
 $= n \log n + n \log n$
 $T(n) = O(n \log n)$

General time complexity is
 Best Average = worst = $O(n \log n)$

CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented and tested. It sorts the elements by creating a max-heap and repeatedly placing the largest element in its correct position. It is efficient for large amounts of data and has a time complexity of $O(n \log n)$.